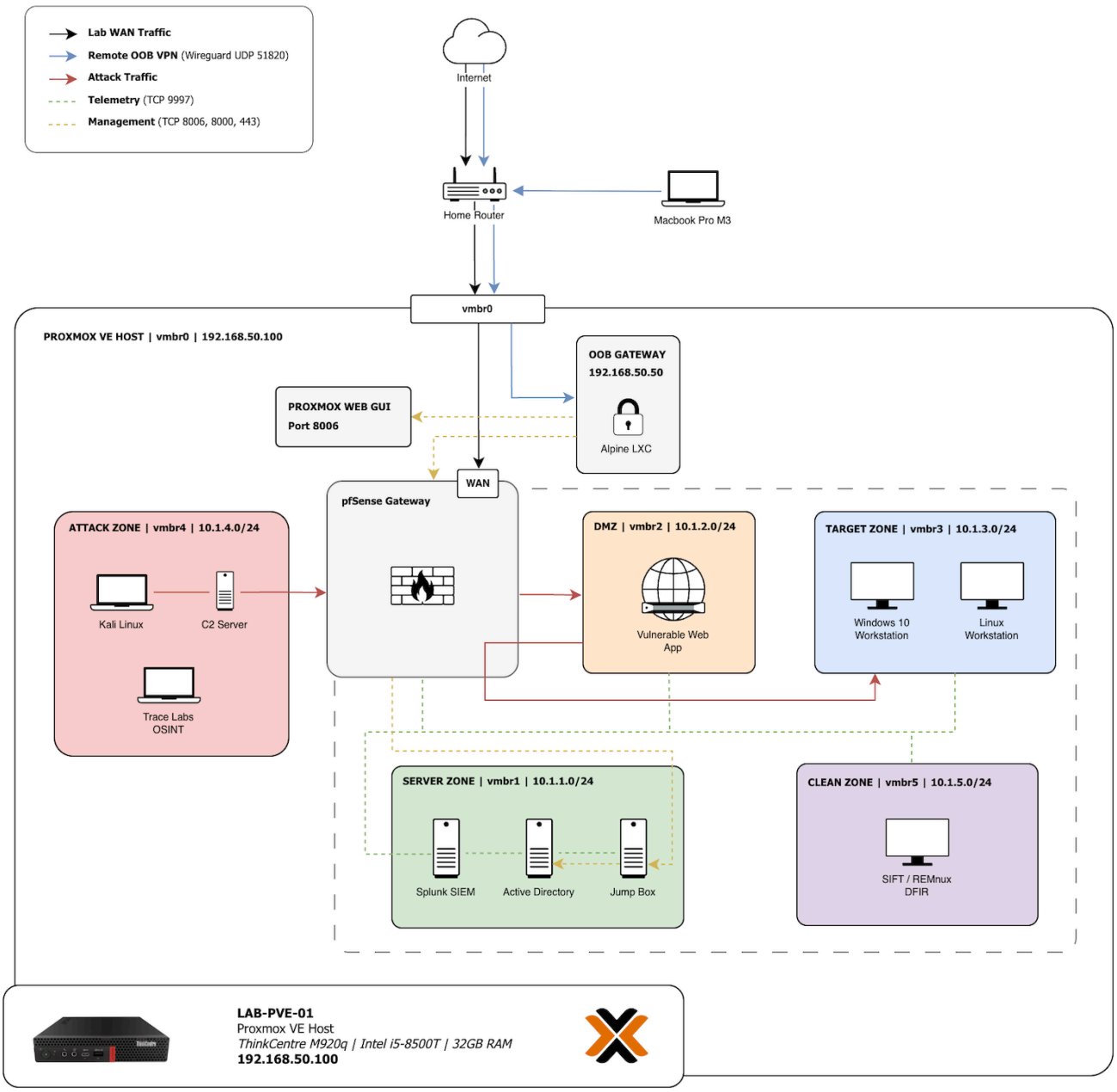
Projekt 1: Bygga ett säkert cybersäkerhetslabb hemma: pfSense, Active Directory och Splunk

Systemarkitektur & mål-topologi



Tillgångsförteckning

Värdnamn	OS	IP-adress	Subnät / Zon	Roll
pfSense	FreeBSD	10.1.x.1	Gateway Core	Central router & policybrandvägg
oob-gateway	Alpine Linux	192.168.50.50	Out-of-Band	WireGuard LXC för fjärråtkomst

Värdnamn	OS	IP-adress	Subnät / Zon	Roll
splunk-siem	Ubuntu Server	10.1.1.10	Server Vault (LAN)	Central loggindexerare & analys
dc01	Windows Server 2022	10.1.1.20	Server Vault (LAN)	Active Directory-domänkontrollant (Telemetri: Sysmon + Splunk Forwarder)
jumpbox-01	Windows 10	10.1.1.50	Server Vault (LAN)	Härdad administrationsarbetsstation
vuln-web	Ubuntu Server	10.1.2.10	DMZ (OPT1)	Sårbar webbapplikation
win10-client01	Windows 10	10.1.3.100	Målzon (OPT2)	Domänansluten Active Directory-klient
linux-client01	Ubuntu Desktop	10.1.3.150	Målzon (OPT2)	Linux-arbetsstation (Telemetri: Splunk Forwarder)
kali-attack	Kali Linux	10.1.4.100	Attackzon (OPT3)	Primär arbetsstation för adversary emulation
c2-server	Ubuntu Server	10.1.4.150	Attackzon (OPT3)	Command and Control-server (C2)
osint	Debian (Trace Labs OSINT)	10.1.4.200	Attackzon (OPT3)	Arbetsstation för målprofilering & OSINT
dfir-remnux	REMnux	10.1.5.100	Ren zon / Clean Zone (OPT4)	Arbetsstation för forensisk undersökning & skadlig kod-analys

Nätverkssegmenteringspolicy

Gränssnitt	Subnät	Gateway	DHCP	Utgående regelpolicy
LAN (Server)	10.1.1.0/24	10.1.1.1	Inaktiverad	Isolerad. Anpassade tillåtelsesregler endast för AD/DNS-synkronisering.
OPT1 (DMZ)	10.1.2.0/24	10.1.2.1	Aktiverad	Internet tillåtet; alla lokala labbzoner blockerade.
OPT2 (Mål)	10.1.3.0/24	10.1.3.1	Aktiverad	Internet tillåtet; alla lokala labbzoner blockerade.
OPT3 (Attack)	10.1.4.0/24	10.1.4.1	Aktiverad	Internet tillåtet; alla lokala labbzoner blockerade.
OPT4 (Ren)	10.1.5.0/24	10.1.5.1	Inaktiverad	Ingen internetåtkomst tillåten.

Modul 1: Infrastruktur och isolering

Hypervisor-konfiguration & Proxmox-lagringsarkitektur

Jag satte upp mitt labb på en liten Lenovo ThinkCentre M920q mini-PC. Först aktiverade jag virtualisering i BIOS, installerade Proxmox VE och bytte till det kostnadsfria community-arkivet för att bli av med licens-popupen.



För att få extra utrymme anslöt jag en extern 1 TB Samsung T7 USB-SSD. Jag formaterade den som ext4 eftersom det skyddar data bättre om USB-kabeln skulle råka kopplas ur. Jag hämtade enhetens UUID med `blkid` och lade till den i min `fstab`-fil med flaggan `nofail` så att Proxmox inte kraschar under uppstart om enheten inte är inkopplad.

ID ↑	Type	Content	Path/Target	Shared	Enabled
local	Directory	Backup, Import, ISO image, Container t…	/var/lib/vz	No	Yes
local-lvm	LVM-Thin	Disk image, Container		No	Yes
samsung-t7	Directory	Backup, Disk image, ISO image, Contai…	/mnt/samsung_t7	No	Yes

Jag delade upp mina lagringssökvägar för att skydda min data. Kritisk kärninfrastruktur som pfSense och Active Directory-domänkontrollanten tilldelades den interna enheten. Jag valde detta eftersom Active Directory-databaser kan korrumperas direkt om en USB-enhet kopplas från ens för en sekund. Sekundära arbetsbelastningar, tunga loggindexerare som Splunk och allmänna arbetsstationer placerades på den externa SSD-enheten för att spara internt utrymme.

Virtual Switch Creation & Central Routing (pfSense)

Skapande av virtuella switchar & central routing (pfSense)

Därefter använde jag Proxmox inställningar för virtuella switchar för att skapa fem virtuella nätverk döpta till `vmbr1` till `vmbr5`. Jag lämnade dessa nätverk helt frånkopplade från mitt fysiska hemnätverk så att ingen trafik kan läcka ut.

I mitten installerade jag pfSense som central router och brandvägg, och kopplade mina bridge-nummer direkt till mina subnät. Jag tilldelade statiska IP-adresser till mina kärnservrar och konfigurerade DHCP för klientzonerna.

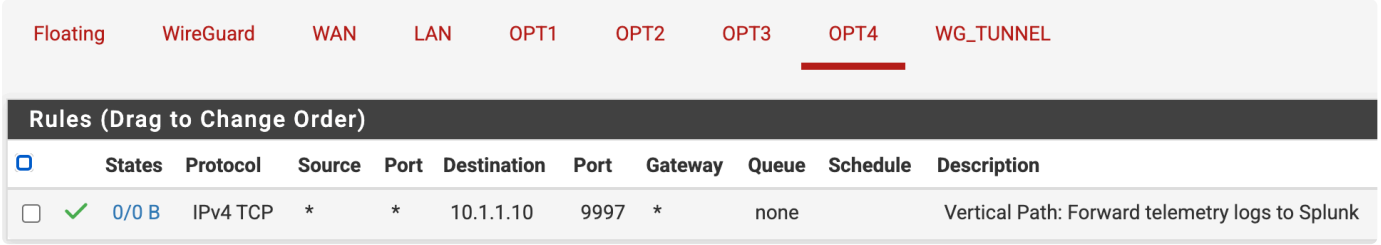
För att säkerställa att skadlig kod inte kan sprida sig till andra nätverk skapade jag en aliaslista i pfSense kallad `All_Private_IPs` som täcker mina lokala subnät. På mina huvudnätverk lade jag till en enkel säkerhetsstruktur med två regler överst i listan:

1. Släpp igenom loggar till min Splunk-server (`10.1.1.10`) på port 9997.
2. Tillåt internetåtkomst samtidigt som allt på min lokala IP-lista blockeras.

Detta tillåt mina VM:ar att hämta uppdateringar men hindrade dem från att kommunicera med varandra.

	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description
☐ ✓	0/0 B	IPv4 TCP	*	*	10.1.1.10	9997	*	none		Forward telemetry logs to Splunk
☐ ✓	0/0 B	IPv4 *	*	*	! All_Private_IPs	*	*	none		Allow internet, block all private networks

Jag aktiverade DHCP på OPT1, OPT2 och OPT3, men höll det inaktiverat på serverzonen (LAN) och skadlig kod-renrummet (OPT4) för att hålla miljön stabil. På gränssnittet för skadlig kod-renrummet (OPT4) tog jag bort internetregeln helt. Genom att bara lämna Splunk-porten öppen förblir aktiv skadlig kod helt offline men kan fortfarande skicka loggar till min SIEM.



Out-of-Band-åtkomstgateway (Alpine WireGuard LXC)

För att hantera labbet på distans utan att exponera det mot internet skapade jag en oprivilegerad Alpine Linux-container som kör WireGuard. På grund av säkerhetsbegränsningarna i oprivilegerade containrar kunde den inte kommunicera direkt med nätverkskorten. Jag upptäckte att jag manuellt behövde redigera containerns konfigurationsfil i Proxmox för att skicka vidare enheten `/dev/net/tun`.

```
root@pve01:~# cat /etc/pve/lxc/101.conf
arch: amd64
cores: 1
features: nesting=1
hostname: OOB-Gateway
memory: 512
net0: name=eth0,bridge=vmbr0,gw=192.168.50.1,hwaddr=BC:24:11:14:E3:C7,ip=192.168.50.50/24,type=veth
onboot: 1
ostype: alpine
rootfs: local-lvm:vm-101-disk-0,size=4G
swap: 512
unprivileged: 1
lxc.cgroup2.devices.allow: c 10:200 rwm
lxc.mount.entry: /dev/net dev/net none bind,create=dir
```

Genom att hålla containern oprivilegerad och bara skicka vidare just denna enhet säkerställde jag att även om en angripare skulle lyckas kompromettera mitt WireGuard VPN, är de helt fångade i containern och kan inte ta sig ut till min fysiska Proxmox-hypervisor.

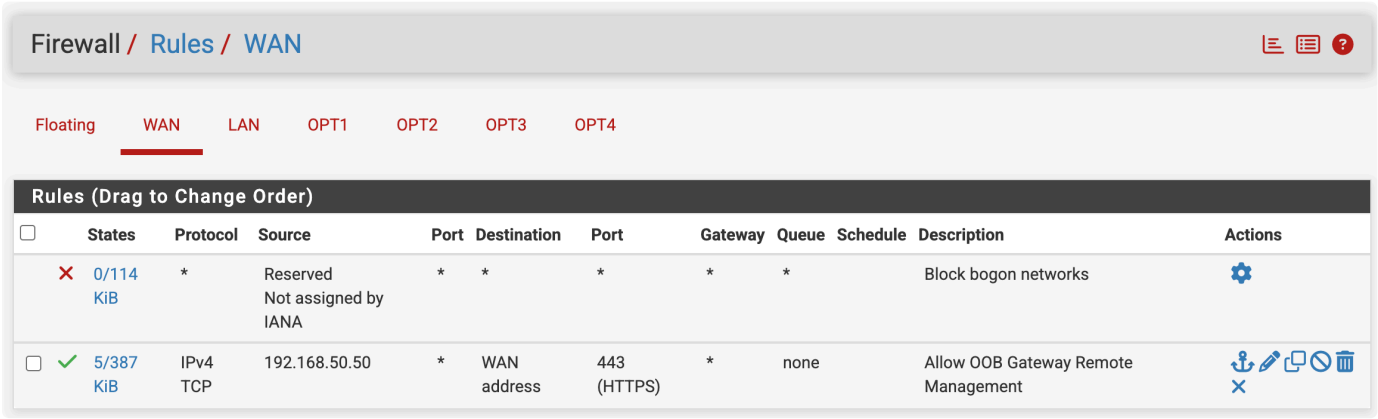
När det var klart vidarebefordrade jag UDP-port 51820 på min hemrouter, konfigurerade ett split-tunnel-VPN och aktiverade NAT-routing med `iptables` i Alpine. Först kunde jag inte nå det interna nätverket från min bärbara dator eftersom WireGuard-containern saknade en rutt tillbaka till mina interna labbsubnät. Genom att lägga till `up ip route add 10.1.0.0/16 via 192.168.50.42` i `/etc/network/interfaces` löstes problemet.

```
OOB-Gateway:~# cat /etc/network/interfaces
auto lo
iface lo inet loopback
iface lo inet6 loopback

auto eth0
iface eth0 inet static
        address 192.168.50.50/24
        gateway 192.168.50.1
        hostname $(hostname)

up ip route add 10.1.0.0/16 via 192.168.50.42
```

Slutligen gick jag in i pfSense och lade till en WAN-brandväggsregel som endast tillåter HTTPS-hanteringsåtkomst från min WireGuard-IP på `192.168.50.50`.



Modul 2: Centrala enterprise-tjänster i valvet

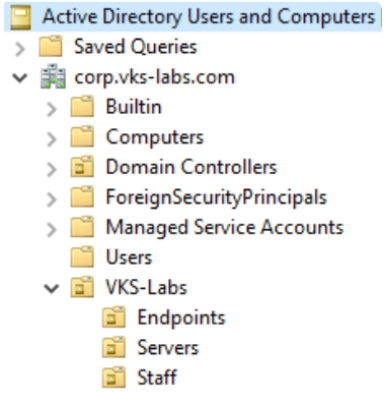
Active Directory-domänkontrollant (dc01)

Jag byggde en Windows Server 2022-domänkontrollant på `10.1.1.20` i Server Vault (LAN). Jag använde standarddiskinställningar så att Windows kunde läsa disken, kopplade nätverkskortet till VirtIO-drivrutinen och stängde av hypervisorns brandvägg så att pfSense hanterar säkerheten.

Eftersom Windows inte har inbyggda VirtIO-drivrutiner startade den helt offline. Jag monterade gästdrivrutins-ISO:n (`virtio-win.iso`) i den virtuella CD-enheten och använde Enhetshanteraren för att installera drivrutinen för nätverkskortet.

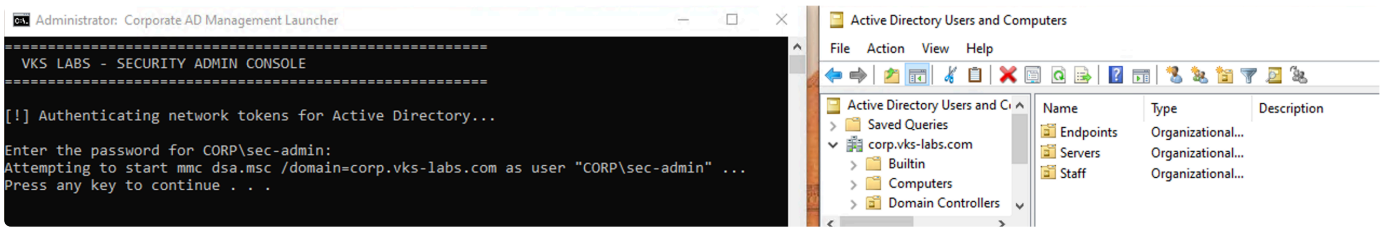


Jag ändrade servernamnet till `dc01` och befordrade den till domänkontrollant för `corp.vks-labs.com` . Jag satte även upp en strukturerad Organisationsenhet (OU) i Active Directory med mapparna *Endpoints*, *Servers* och *Staff*.



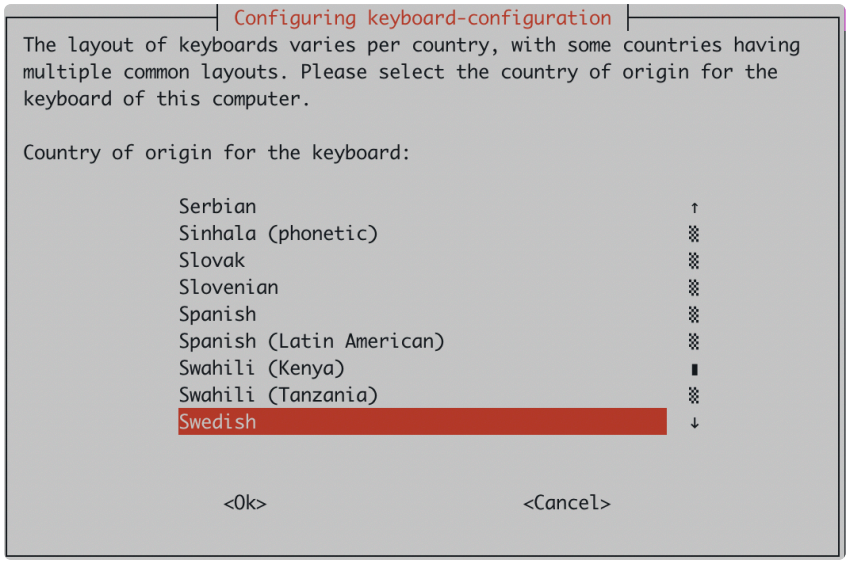
Härdad administrations-jumpbox (jumpbox-01)

Jag konfigurerade en Windows Jump Box på `10.1.1.50` men lämnade den oansluten från domänen. Detta säkerställer att Domain Admin-lösenord aldrig sparas i minnet där en angripares verktyg kan komma åt dem. Jag använde ett anpassat skript för att säkert hantera Active Directory från denna oanslutna arbetsstation och använde fjärrskrivbord mot domänkontrollanten för ändringar i Group Policy.



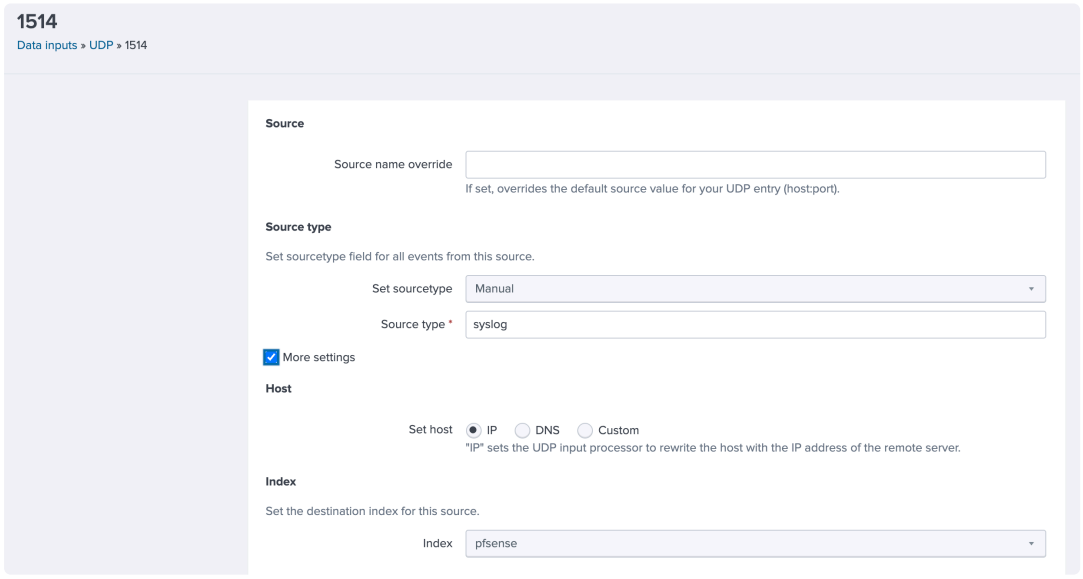
Central loggindexering (splunk-siem)

Jag installerade min huvudsakliga Splunk SIEM-server på en ren Ubuntu Server-instans på `10.1.1.10` i Server Vault. Direkt kunde jag inte logga in på konsolen eftersom Ubuntus installationsprogram hade valt amerikansk tangentbordslayout som standard, vilket misstämde de specialtecken som finns på mitt fysiska svenska tangentbord. Jag klistrade in lösenordet över en aktiv SSH-session för att komma in och korrigerade därefter systemets tangentbordsinställningar.



När jag försökte starta Splunk kraschade det direkt eftersom uppackning av filerna med root-rättigheter gjorde att mapparna ägdes av root. En snabb `chown` löste mappbehörigheterna.

Inne i Splunk skapade jag ett brandväggsindex begränsat till 100 GB, samt dedikerade Windows- och Linux-index begränsade till 50 GB vardera. Eftersom Linux blockerar vanliga konton från att använda portar under 1024, konfigurerade jag min Splunk syslog-lyssnare på port 1514 istället.



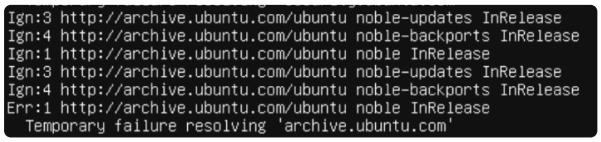
I pfSense aktiverade jag fjärrloggning och pekade den mot `10.1.1.10:1514`, vilket direkt fyllde min instrumentpanel med brandväggshändelser.



Modul 3: Målmiljöer & telemetri-onboarding

Sårbar webbapplikation i DMZ (vuln-web)

Jag skapade en Ubuntu Server-VM döpt till `vuln-web` på `10.1.2.10` i mitt DMZ-nätverk (`vmbr2`), och utökade den logiska rotvolymen (`ubuntu-lv`) för att utnyttja hela diskutrymmet. I slutet av installationen hängde sig processen på säkerhetsuppdateringar på grund av DNS-uppslagsfel för `archive.ubuntu.com`. Jag insåg att pfSense blockerade DMZ-trafik från att nå min Active Directory DNS-server på `10.1.1.20` på LAN, så jag hoppade över uppdateringarna och startade om operativsystemet.



För att lösa nätverket permanent redigerade jag `/etc/netplan/50-cloud-init.yaml` för att ersätta DHCP med en statisk IP (`10.1.2.10/24`), satte gateway till `10.1.2.1` och pekade DNS-servrarna direkt mot `10.1.2.1` och `1.1.1.1`. Körning av `sudo netplan apply` återställde internetåtkomsten.

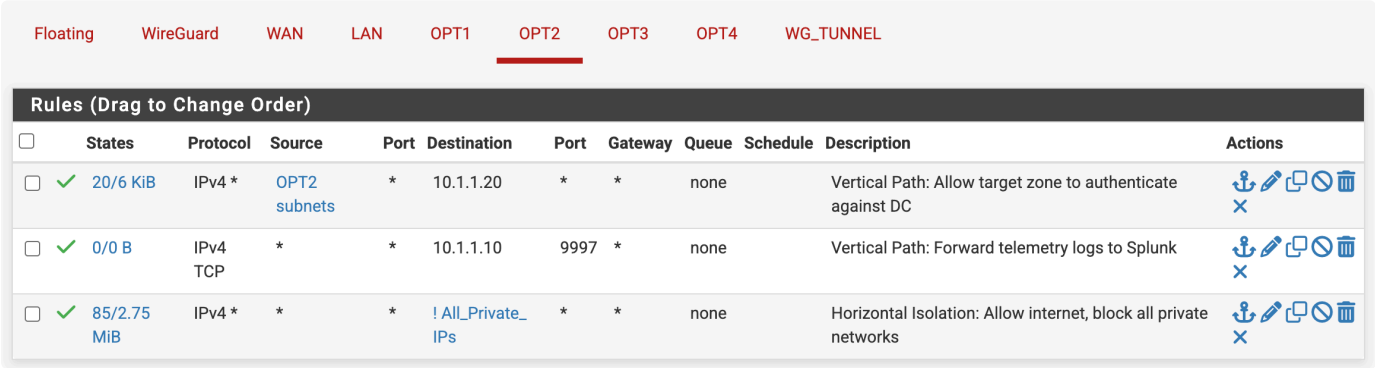


Efter att ha återanslutit till `10.1.2.10` över WireGuard installerade jag `qemu-guest-agent` för Proxmox-rapportering, installerade Docker och startade DVWA-containern (`vulnerables/web-dvwa`) på port 80.

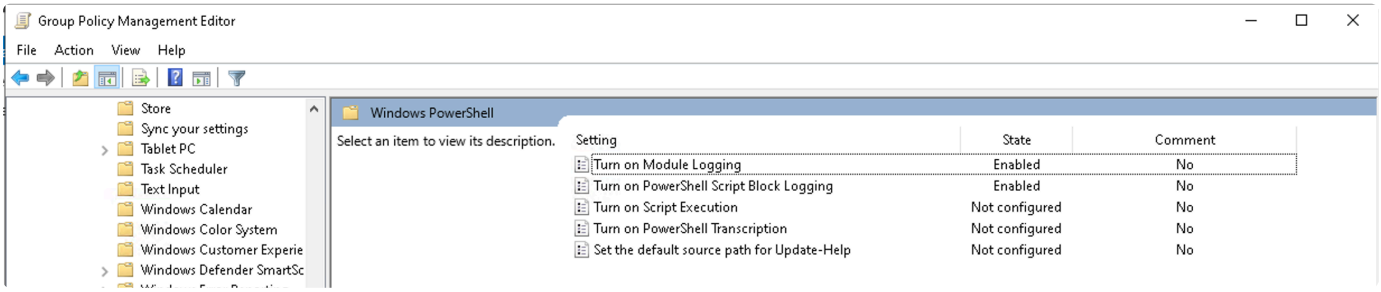


Onboarding & felsökning av målslutpunkter (win10-client01)

Jag distribuerade en Windows 10-klient (`win10-client01`) på `10.1.3.100` på OPT2. När jag försökte ansluta den till domänen tajmade anslutningen ut. Jag spårade detta till min isoleringsregel i brandväggen som blockerade lokal trafik mellan subnät. Genom att dra en pass-regel för AD-autentisering överst i min brandväggsregellista för OPT2 kunde slutpunkten nå `10.1.1.20` , vilket löste problemet.



Därefter skapade jag en Group Policy i domänen för att aktivera *PowerShell Script Block Logging* och *Module Logging*.



Klienten misslyckades dock initialt med att hämta denna policy. Jag upptäckte att det fanns en tidsavvikelse på 9 timmar mellan värddatorns moderkortsklocka och domänkontrollanten. Eftersom Active Directory avvisar alla tidsskillnader över 5 minuter tvingade jag klienten att synkronisera med domänkontrollanten via `net time \\10.1.1.20 /set /y` och korrigerade serverns tidszon.

```
PS C:\Windows\system32> net time \\10.1.1.20 /set /y
Current time at \\10.1.1.20 is 2026-07-06 19:03:06

The command completed successfully.

PS C:\Windows\system32>
PS C:\Windows\system32> gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
User Policy update has completed successfully.
```

Även när tiden var synkroniserad dök inte testkommandon upp i Splunk. En policyrapport visade att klientens datorobjekt låg i standardmappen *Computers*, som inte stöder policyarv. Jag drog datorobjektet till min *Endpoints*-OU, döpte om datorn till `win10-client01` och startade om.

Telemetripipeline för slutpunkter (Sysmon, GPO & Forwarders)

För att skicka Windows-händelser till Splunk installerade jag Splunk Universal Forwarder på `win10-client01`. Eftersom standardloggarna i Windows innehåller mycket brus installerade jag Microsoft Sysmon och tillämpade Olaf Hartongs konfiguration för att filtrera bort bakgrundsaktivitet och fånga processskapande samt nätverksanslutningar. Därefter öppnade jag forwarderns `inputs.conf`-fil och lade till sökvägar för Application-, Security-, System-, Sysmon- (`Microsoft-Windows-Sysmon/Operational`) och PowerShell-loggar (`Microsoft-Windows-PowerShell/Operational`).

inputs - Notepad

File Edit Format View Help

```
[WinEventLog://Application]
index = windows
disabled = false

[WinEventLog://Security]
index = windows
disabled = false

[WinEventLog://System]
index = windows
disabled = false

[WinEventLog://Microsoft-Windows-Sysmon/Operational]
index = windows
disabled = false
renderXml = true
sourcetype = XmlWinEventLog:Microsoft-Windows-Sysmon/Operational


[WinEventLog://Microsoft-Windows-PowerShell/Operational]
index = windows
disabled = false
renderXml = true
```

Därefter konfigurerade jag telemetri på min domänkontrollant (`dc01`). Jag installerade Sysmon64 med Olaf Hartongs konfiguration. När jag körde installationsprogrammet för Splunk Universal Forwarder på domänkontrollanten rullades installationen tillbaka och kraschade upprepade gånger. Jag upptäckte att eftersom en domänkontrollant saknar en lokal användardatabas misslyckas installationsprogrammets standardinställning "Virtual Account" när det försöker lägga till sig själv i lokala prestandaövervakningsgrupper. Genom att ändra tjänstekontot för installationen till "Local System" löstes problemet direkt, vilket tillät installationen att slutföras och gav forwardern de behörigheter som krävdes för att läsa skyddade katalogloggar.

The user you install UniversalForwarder as determines what data it has access to. The Managed Service Account and Group-Managed Service Account are supported by CLI only.

Install UniversalForwarder as:

- (•) Local System
  Installs UniversalForwarder using local system account. UniversalForwarder can access all data on or forwarded to this machine.
- ( ) Domain Account
  Installs UniversalForwarder with domain account you provide. This lets you collect logs and metrics from remote machines as well as local and forwarded data. You can set the account in the next dialog, as a local administrator or a reduced privilege user.
- ( ) Virtual Account
  Installs UniversalForwarder using a virtual account. UniversalForwarder can access all data on or forwarded to this machine.

När loggarna från domänkontrollanten började strömma in märkte jag att den genererade enorma loggvolymer från Windows Filtering Platform-anslutningar och filhantering, vilket hotade min kostnadsfria dagliga Splunk-licensgräns på 500 MB. För att lösa detta redigerade jag `inputs.conf` på domänkontrollanten och lade till en exkluderingsregel (blacklist):

```
[WinEventLog://Security]
index = windows
disabled = false
blacklist1 = EventCode="(5156|5158|4656|4658|4690|4907)"
```

En omstart av forwardern minskade de upprepade bakgrundshändelserna avsevärt medan kritiska inloggnings- och policyändringsloggar bevarades.

Härdning av Linux-slutpunkt (linux-client01)

På min Linux-målmaskin (`10.1.3.150`) märkte jag att Splunk-forwardern kördes som root. Jag konfigurerade om tjänsten till att köras under ett begränsat `splunkfwd` -tjänstekonto. För att tillåta Splunk att samla in bash-kommandohistorik utan att öppna privata användarmappar för alla, använde jag Access Control Lists (`setfacl`) för att ge läsrättigheter enbart till `splunkfwd` :

```
sudo setfacl -m u:splunkfwd:r /home/nix-localadmin/.bash_history
```

Detta tillät kommandologgar att strömma säkert till `linux` -indexet samtidigt som slutpunkten förblev ordentligt härdad.

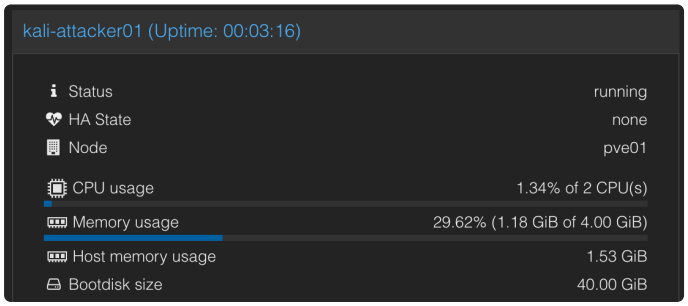
```
nix-localadmin@nix-client01:~$ ps aux | grep splunkd
splunkf+   26723  0.5  8.4 370344 142912 ?       Ssl  11:32   0:00 splunkd --under-systemd --systemd-delegate=no -p 8089
 _internal_launch_under_systemd
splunkf+   26778  0.0  0.8 169776 14948 ?        Ss   11:32   0:00 [splunkd pid=26723] splunkd --under-systemd --systemd
-delegate=no -p 8089 _internal_launch_under_systemd [process-runner]
nix-loc+   26843  0.0  0.1  18408  2464 pts/0    S+   11:34   0:00 grep --color=auto splunkd
```

Modul 4: Adversary Emulation & säkerhetsvalidering

Distribution av attackinfrastruktur

1. Primär attackarbetsstation (kali-attacker01)

Jag installerade min primära Kali Linux-maskin på `10.1.4.100` i Attackzonen (`vmbr4`) för att hantera aktiv adversary testing. Jag aktiverade även SSH och installerade `qemu-guest-agent` .



2. Command and Control-server (c2-server)

Jag satte upp en Ubuntu Server-VM på `10.1.4.150` på `vmbr4` . Under installationen satte jag en statisk IP (`10.1.4.150/24`), aktiverade SSH och utökade disken till att använda hela utrymmet på 30 GB. När jag väl var inloggad installerade jag `qemu-guest-agent` , uppdaterade OS, installerade `mingw-w64` för att korskompilera Windows-payloads och installerade Sliver C2-ramverket.

```
attacker@c2-server:~$ sudo systemctl status qemu-guest-agent
● qemu-guest-agent.service - QEMU Guest Agent
     Loaded: loaded (/usr/lib/systemd/system/qemu-guest-agent.service; static)
     Active: active (running) since Tue 2026-07-28 12:59:27 UTC; 20s ago
   Main PID: 1419 (qemu-ga)
      Tasks: 2 (limit: 2263)
     Memory: 528.0K (peak: 932.0K)
        CPU: 12ms
     CGroup: /system.slice/qemu-guest-agent.service
             └─1419 /usr/sbin/qemu-ga
```

```
attacker@c2-server:~$ sliver
Connecting to 127.0.0.1:31337 ...

All hackers gain undying
[*] Server v1.7.3 - 3bbaf805104dcc4a75414ee0084e8de50702cad4
[*] Welcome to the sliver shell, please type 'help' for options

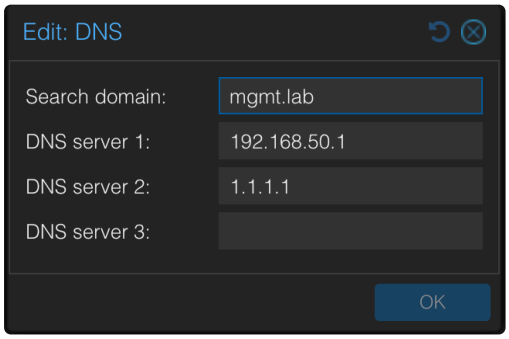
[*] Check for updates with the 'update' command
```

3. Arbetsstation för målprofilering (osint)

Jag installerade Trace Labs OSINT-VM på `10.1.4.200` på `vmbr4` för att hålla målprofilering skild från min aktiva attacktrafik. När jag laddade ner OVA-filen misslyckades Proxmox med att slå upp `github.com`.

```
root@pve01:/var/lib/vz/dump# wget https://github.com/tracelabs/tlosint-vm/releases/download/2026.07/tl-osint-2026.7-vmware-amd64.ova
--2026-07-28 15:04:52--  https://github.com/tracelabs/tlosint-vm/releases/download/2026.07/tl-osint-2026.7-vmware-amd64.ova
Resolving github.com (github.com)... failed: Temporary failure in name resolution.
wget: unable to resolve host address ‘github.com’
root@pve01:/var/lib/vz/dump#
```

Jag uppdaterade värddatorns DNS-inställningar i Proxmox GUI för att inkludera Cloudflare (`1.1.1.1`) som sekundär DNS-server.



Efter att ha laddat ner arkivet packade jag upp innehållet, skapade ett tomt VM-skal utan ansluten lagring och importerade den extraherade disken direkt till min Proxmox-lagringspool:

```
qm importdisk 110 tl-osint-2026.7-vmware-amd64-disk1.vmdk samsung-t7
```

Jag anslöt den importerade disken som `scsi0` , satte den som boot-enhet och startade VM:en.

Eftersom `openssh-server` inte var inkluderat som standard öppnade jag konsolen, installerade SSH, aktiverade tjänsten, ändrade standardanvändarens lösenord och installerade `qemu-guest-agent` .

```
osint@tlosint:~$ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/system
d/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
```

4. Forensik- och skadlig kod-renrum (dfir-remnux)

Jag installerade en REMnux-VM (VM 111) på `10.1.5.100` i OPT4 Clean Zone (`vmbr5`) för att skapa en isolerad miljö för reverse engineering och analys av skadlig kod.

När jag först försökte starta SSH-tjänsten (`sudo systemctl enable --now ssh`) misslyckades det. Jag upptäckte att rena REMnux-installationer rensar SSH-värdnycklar som standard så att varje installation får unika nycklar. Jag löste detta genom att generera nya nycklar med `sudo ssh-keygen -A` , vilket gjorde att SSH kunde starta utan problem.

```
remnux@remnux:~$ sudo ssh-keygen -A
remnux@remnux:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-07-31 07:33:54 UTC; 44s ago
```

Jag ändrade även tangentbordslayouten från engelska till svenska.

För att konfigurera telemetri medan VM:en fortfarande hade internetåtkomst anslöt jag den tillfälligt till `vmbr4` för att ladda ner Splunk Universal Forwarder-paketet. Jag installerade forwardern, godkände licensen, ställde in den att starta vid boot och pekade den till att skicka loggar direkt till min Splunk-server på `10.1.1.10:9997` .

Därefter konfigurerade jag en statisk IP (`10.1.5.100/24`) med gateway `10.1.5.1` i Netplan (`/etc/netplan/01-netcfg.yaml`).

```
  GNU nano 7.2                                  /etc/netplan/01-netcfg.yaml *
network:
  version: 2
  ethernets:
    ens18:
      dhcp4: no
      addresses:
        - 10.1.5.100/24
      routes:
        - to: default
          via: 10.1.5.1
```

Netplan gav en behörighetsvarning, så jag begränsade åtkomsten till filen med `sudo chmod 600 /etc/netplan/01-netcfg.yaml` .

```
remnux@remnux:~$ sudo netplan apply

** (generate:2831): WARNING **: 07:52:03.518: Permissions for /etc/netplan/01-netcfg.yaml are too open. Netplan configur
ation should NOT be accessible by others.
```

```
remnux@remnux:~$ sudo chmod 600 /etc/netplan/01-netcfg.yaml
```

För att rensa den gamla DHCP-adressen (`10.1.4.103`) körde jag `sudo ip addr flush dev ens18` innan jag tillämpade Netplan igen.

Slutligen ändrade jag VM:ens nätverkskort tillbaka till `vmbr5` i Proxmox och aktiverade QEMU Guest Agent (`qm set 111 --agent enabled=1`) för att möjliggöra snygga OS-nedstängningar, live-IP-rapportering och säkra snapshots.

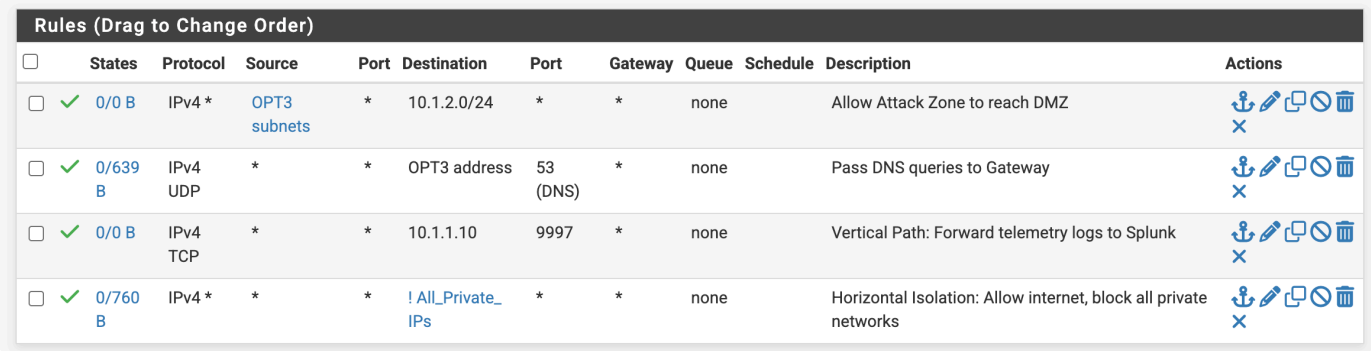
```
remnux@remnux:~$ sudo systemctl status qemu-guest-agent
● qemu-guest-agent.service - QEMU Guest Agent
     Loaded: loaded (/usr/lib/systemd/system/qemu-guest-agent.service; static)
     Active: active (running) since Fri 2026-07-31 07:57:12 UTC; 1min 1s ago
```

Isolering av zongränser & säkerhetsvalideringstestning

Jag körde nätverksvalideringstester över Attackzonen (`vmbr4`) och Clean Zone (`vmbr5`) för att bekräfta att mina brandväggsregler i pfSense fungerade korrekt.

1. Gränskontroller i Attackzonen

Från Kali (`10.1.4.100`) tajmade en initial HTTP-förfrågan till DMZ-webbservern (`curl -I [http://10.1.2.10](http://10.1.2.10)`) ut eftersom min standardisoleringsregel blockerar trafik mellan subnät. Jag lade till en pass-regel överst i brandväggsgränssnittet för OPT3 för att tillåta HTTP/HTTPS-trafik till `10.1.2.0/24` .



Rules (Drag to Change Order)

	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
✓	0/0 B	IPv4 *	OPT3 subnets	*	10.1.2.0/24	*	*	none		Allow Attack Zone to reach DMZ	
✓	0/639 B	IPv4 UDP	*	*	OPT3 address	53 (DNS)	*	none		Pass DNS queries to Gateway	
✓	0/0 B	IPv4 TCP	*	*	10.1.1.10	9997	*	none		Vertical Path: Forward telemetry logs to Splunk	
✓	0/760 B	IPv4 *	*	*	! All_Private_IPs	*	*	none		Horizontal Isolation: Allow internet, block all private networks	

En omtestning med `curl -I [http://10.1.2.10](http://10.1.2.10)` returnerade direkt ett `HTTP/1.1 302 Found` -svar.

```
┌──(attacker㉿kali-attacker01)-[~]
└─$ curl -I http://10.1.2.10
HTTP/1.1 302 Found
Date: Wed, 22 Jul 2026 11:19:34 GMT
Server: Apache/2.4.25 (Debian)
Set-Cookie: PHPSESSID=6j9vm38j0m3a8a1r4ue371c2k2; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=6j9vm38j0m3a8a1r4ue371c2k2; path=/
Set-Cookie: security=low
Location: login.php
Content-Type: text/html; charset=UTF-8
```

Därefter testade jag anslutningar från Kali till domänkontrollanten (`10.1.1.20`) och Splunk-servern (`10.1.1.10`) i Server Vault. Pinging av `10.1.1.20` resulterade i 100 % paketförlust, och `nc -zvw3 10.1.1.10 8000` tajmade ut helt. Detta bekräftade att pfSense tillåter min attackmaskin att nå exponerade DMZ-tjänster samtidigt som lateral förflyttning till känsliga interna nätverk blockeras.

```
┌──(attacker㉿kali-attacker01)-[~]
└─$ ping -c 3 10.1.1.20
PING 10.1.1.20 (10.1.1.20) 56(84) bytes of data.

--- 10.1.1.20 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2051ms


┌──(attacker㉿kali-attacker01)-[~]
└─$ nc -zvw3 10.1.1.10 8000
10.1.1.10: inverse host lookup failed: Unknown host
(UNKNOWN) [10.1.1.10] 8000 (?) : Connection timed out
```

2. Isolering i Clean Zone & kontroller av telemetrieffektivitet

Från `dfir-remnux` (`10.1.5.100`) körde jag två nätverkskontroller för att bekräfta säkerheten i renrummet för skadlig kod:

Körning av `ping -c 3 1.1.1.1` resulterade i 100 % paketförlust. Detta bekräftade att utgående internetåtkomst är helt blockerad på `vmbr5`, vilket hindrar exekverad skadlig kod från att ansluta till externa Command and Control-servrar (C2).

```
remnux@remnux:~$ ping -c 3 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.

--- 1.1.1.1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2086ms
```

Körning av `nc -zvw3 10.1.1.10 9997` returnerade `[10.1.1.10] 9997 (?) open`. Detta verifierade att trots att VM:en är avskuren från internet kan den fortfarande skicka loggar till Splunk-indexeraren.

```
remnux@remnux:~$ nc -zvw3 10.1.1.10 9997
10.1.1.10: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [10.1.1.10] 9997 (?) open
```

Konvertering av lagringsformat & baslinje-snapshots

För att spara labbets rena baslinjetillstånd innan exekvering av angreppssimuleringar eller skadlig kod skapade jag ögonblicksbilder (snapshots döpta till `Clean-Base`) över alla VM:ar.

När jag försökte ta en snapshot av `dfir-remnux` gav Proxmox ett felmeddelande: *"The current guest configuration does not support taking new snapshots"*. Körning av `qm config 111 | grep scsi` på `pve01` visade att disken var lagrad som en `.raw`-avbildning (`vm-111-disk-0.raw`) på kataloglagringspoolen (`samsung-t7`). Jag upptäckte att RAW-filer på kataloglagring inte stöder snapshots.

```
root@pve01:~# qm config 111 | grep scsi
boot: order=scsi0
scsi0: samsung-t7:111/vm-111-disk-0.raw,iothread=1,size=100G
scsihw: virtio-scsi-single
```

Jag konverterade disken till QCOW2 via Proxmox CLI:

```
qm disk move 111 scsi0 samsung-t7 --format qcow2 --delete 1
```

Detta konverterade disken till `vm-111-disk-0.qcow2` och raderade den gamla `.raw`-filen för att spara utrymme.

När jag tog baslinje-snapshots över `sec-pfsense`, `sec-splunk`, `win10-client01`, `kali-attacker01` och `dfir-remnux` avmarkerade jag avsiktligt *Include RAM*. Genom att exkludera aktivt RAM-minne hålls snapshot-storlekarna små, återställningar går avsevärt snabbare och VM:arna garanteras starta upp fräsch utan låsta nätverksanslutningar.

Projektresultat & nästa steg

Med Projekt 1 slutfört har jag nu ett fullt fungerande och säkert cybersäkerhetslabb redo för användning. Allt för routing, lagring, Active Directory och logginsamling är uppsatt och testat:

- **Centraliserad routing & isolering:** Jag konfigurerade pfSense för att hantera fem isolerade virtuella nätverk (`vmbr1` – `vmbr5`). Detta tillåter mina VM:ar att nå internet vid behov, samtidigt som mitt REMnux-renrum för skadlig kod (OPT4) hålls helt offline.
- **Logginsamlingspipeline:** Min Splunk-server (`10.1.1.10`) samlar aktivt in brandväggsloggar från pfSense, Sysmon- och Windows Event-loggar från min domänkontrollant och klient, samt bash-kommandologgar från Linux-slutpunkter.

- **Säkerhetsvalidering:** Jag har verifierat att mina attackverktyg på OPT3 kan nå den sårbara webbservern i DMZ utan att kunna nå några av mina kärnservrar på det interna nätverket. Renrummet för skadlig kod förblir isolerat från nätet, men kan fortfarande strömma loggar direkt till Splunk på port 9997.
- **Snabba återställningar:** Att alla VM-diskar konverterats till QCOW2 gör att jag kan ta snabba `Clean-Base`-snapshots utan att slösa extra diskutrymme på minnesdumpar.

Nu när labbet är helt segmenterat, övervakat och säkrat med rena snapshots är jag redo för **Projekt 2: Automatiserad angreppssimulering & SIEM-detekteringar**. Mina nästa steg blir att bygga egna Python-skript för att avfyra automatiserade attacklaster (webbexploits och dold DNS-exfiltrering), automatiskt validera mina Splunk-detekteringsregler via API samt testa post-exploateringsbeteenden med Sliver C2 och Atomic Red Team.